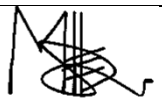
VAAL UNIVERSITY OF TECHNOLOGY

Inspiring thought. Shaping talent.

Faculty: Applied and Computer Sciences
Department: Computer Sciences
Subject: Business Analysis 3.2
Subject Code: AIBUY3A
Lecturer : Mr M Matsela
Moderator : Mr XP Piyose
Year : 2026

Assessment Name:	Project Documentation –AgriScan-AI
Assessment Date:	02 November 2026

STUDENT NUMBER	SURNAME AND INITIALS	SIGNATURE
224632132	MAVANGWA AS	
224725106	MATHEBULA NP	
223076988	RM KUBAYI	
224573594	MT MKHONTO	

DECLARATION

We,

ANDZANI SINGH MAVANGWA
NHLAYISEKO PRIDE MATHEBULA
RISIMA MARVIN KUBAYI
MIKHENSO TRUST MKHONTO

Declare that:

- The contents of this project represent our own unaided work, and the project has not previously been submitted for academic examination towards any qualification.
- We have not engaged in plagiarism or the use of unauthorized materials within our work.
- All resources used in this work have been appropriately cited and acknowledged.
- This work represents our own opinions and not necessarily those of the Vaal University of Technology.



Signed — 02 NOVEMBER 2026 — Date



Signed — 02 NOVEMBER 2026 — Date



Signed — 02 NOVEMBER 2026 — Date



Signed — 02 NOVEMBER 2026 — Date

Table of Contents

DOCUMENTATION ASPECTS

1. AgriScan AI's Relevance to The Theme 4
2. Business Objectives 4
3. Business Success Criteria 5
4. Business Background 5
5. Requirements 5
6. Constraints…. 6
7. Risks…. 6-7
8. Tools and Techniques 8-9
9. Problem Definition 10

THEORETICAL ASPECTS 11

1. Machine Learning Approach 11-13
2. Data 14-15
3. Model 16-18
4. Solution Techniques 18-19
5. Deep Learning 20-22
6. Other Features 23-24

Reference 24

Table of Figures

Figure 1: Multi-task image classification task ..... 11 Figure 2: The model was implemented using a Convolutional Neural Network (CNN) 11

Figure 3: The model uses the PlantVillage dataset…. 12

Figure 4: Outputs for each data class via one of the four classes…… 13-14

Figure 5: The PlantVillage dataset was sourced from the Kaggle dataset library and focused on the plant foliar disease classification task 14

Figure 6: 20% of the training data was set aside for validation 14

Figure 7: In the codebase, all images were resized to 128 X 128 pixels to improve efficiency and normalized to [0, 1] ……. 15

Figure 8: Implementation of a 4-block system of Convolutional and Pooling layers 16

Figure 9: A Flatten layer was used to condense the outputs 17

Figure 10: The model was then set to train for 6 epochs 17

Figure 11: The model was then used to evaluate and p 18

Figure 12: The predict_image function accepts multiple input types and displays comprehensive results including disease

information, confidence scores, and reliability indicators 22

DOCUMENTATION ASPECTS

AgriScan AI's Relevance to The Theme

Diagnosing plant foliar diseases is an important task in agriculture. Even though it is important, it is still a process that takes many resources and expert knowledge to move from just looking to an exact diagnosis. Farmers and extension workers normally look at leaves by hand. This hand_looking method can easily make mistakes. It can be biased even in well_developed areas. Our proposed solution uses the PlantVillage dataset. This dataset is quality and has been checked by experts in plant pathology and agriculture.It holds leaf images. We use it to make a tool that helps farmers and extension officers find and name plant diseases. The tool uses learning to look at pictures and decide what disease is shown.It can detect 38 types of disease such as Apple scab, Cedar apple rust, Corn common rust, Potato early blight, Tomato bacterial spot and many more,on different crops. This AgriScan AI also follows the aim to useAI to bring new ideas and solutions to farming.This will lower the amount of work needed to diagnose. It will also speed up the time from finding a disease to fixing it.That helps keep farms sustainable and food safe.

2. Business Objectives

- The goal is to make plant disease diagnosis happen in seconds of days.
- The aim is to make plant disease diagnosis accurate by using the PlantVillage dataset that has 38 types of diseases.
- The purpose is to help increase crop yields and improve the lives of farmers by finding problems and making better decisions in farming.
- The idea is to make farming cheaper for farmers and agricultural groups without making decisions worse by using less pesticide than before.
- The goal is to show how AI tools like AgriScan AI can help with farming tasks and make expert knowledge available to people.

• The goal is to get people in remote and low-income farming areas in South Africa and around the world to use AI tools like AgriScan AI.

•

3. Business Success Criteria

• The system needs to achieve more than 85% accuracy when classifying plant leaf images into 38 disease types across many crops.

• The system should process 1000 test images and generate a confusion matrix and a classification report in under 5 minutes on a computer with less than 16 GB of RAM.

• The system should analyze a single plant leaf image and identify the disease along with a confidence score in less than 3 seconds.

• The system needs to be used by more than 70% of farmers and agricultural officers who take part in testing.

• The system should provide information about each disease, such as symptoms, how to stop it, and what to do about it.

4.Business Background

More people are needing food and better farming that lasts. Keeping crops healthy has become very important. Because of this need South Africa and other developing countries do not have experts in plant diseases and farming help. Studies say there are not plant doctors to help small farmers, especially in the countryside. These shortages, plus not having money or tools make it hard to find out about plant diseases quickly. This can cause a lot of crops to be lost according to research that says up to 40% of crops are lost every year because of pests and diseases. That costs over $220 billion around the world.We at AgriScan AI see these problems in farming in South Africa and over the world. So, we made an AI tool that looks at pictures of plant leaves to find and name diseases. This helps reduce the work for farming experts. By using machine learning AgriScan AI gives fast and clear answers with confidence levels. This helps farmers and farming experts deal with problems faster. It also helps make farming better by using pesticides, which is better for the environment and helps crop yields grow.

5.Functional requirements

- The system will take pictures of plant leaves in formats like JPG and PNG.
- The system must make the pictures the same size. Prepare them before looking at them.
- The system must say which of the 38 disease types each leaf picture is and also show a confidence score.
- The system will show the 3 guesses for each leaf picture along with the confidence levels.
- The system will make reports about the test data, including a confusion matrix and a classification report.
- The system will give information about each disease, like what the signs are, how to stop it, and what to do about it.

Non-Functional requirements

- The system must be able to look at least 30 leaf pictures every hour at the same time.
- The system must work the way for pictures of leaves in different lighting and quality.The system must be able to add crops and disease types in the future.

•The system must be available. Working 99% of the time during the growing season.

The system must have a command-line interface and an API that makes it easy to use with other farming systems.

6.Constraints

- The system is for leaf diseases. It does not address diseases that affect roots, stems, or fruits, or other farming tasks.
- The system can only look at 38 disease types from the PlantVillage dataset now.
- The model is only tested on data that looks like the PlantVillage dataset.
- Testing will be done on a computer with less than 8 GB of RAM.
- The system needs bright pictures of leaves to work best and may not work as well on lowquality pictures.
- The system is in English for disease information and for the user interface

7.Risks

RISK	LIKELIHOOD	IMAPCT	MITIGATION
Misclassification of diseases	Medium	High	Make sure every image is normalized and sized correctly.
Over-reliance on AI	Medium	High	Include a disclaimer that the system is only for decision support.
			Provide a reporting system with a confidence score to avoid blind trust. Encourage field verification for every prediction.
Poor image quality	High	Medium	Give image quality guidelines to users. Implement checks for image quality before making predictions. Suggest retaking images with better lighting.
Dataset bias	Medium	Medium	Use class weighting to address imbalance. Apply data augmentation such as flips, zooms, and contrast adjustments. Retrain the model with new data.
Technical failure	Medium	High	Perform model backups and version control. Maintain compatibility with TensorFlow versions. Monitor system performance continuously.
Poor User Adoption	Medium	High	Conduct a two -week pilot to teach users about the system. Appoint a champion from agricultural extension services.

Table 1: Risk table

8.Tools And Techniques

A. Programming Language

• Python – the language chosen because it offers many AI libraries, a large community, and is simple to deploy

B. Machine Learning / Deep Learning techniques

• Supervised image classification using Convolutional Neural Networks with the Keras Sequential API

• Data augmentation such as horizontal flips, zooms, and contrast adjustments applied to training data

• Normalize all image data to the [0 1] range

• Use class weighting to address dataset imbalance
• Apply dropout regularization to reduce overfitting

• Implement early stopping to avoid overtraining

C. Python Libraries Used

• os – for file system operations and path checks on the local machine

• Json – for saving and loading model metadata and disease information

• argparse – for command-line argument parsing

• random – for reproducibility by setting a seed

• numpy – for numerical array computations

• tensorflow – the backbone for Keras and input pipelines

• PIL (Python Imaging Library) – for loading and preprocessing images

• matplotlib – to display training history plots and confusion matrix visualizations

• Keras – for defining models, adding layers, training, predicting, and tracking metrics

• sklearn – for generating evaluation reports like confusion matrices, classification reports, and computing class weights

D.Development and collaboration tools

• Meetings / WhatsApp – to discuss the project and share ideas and resources

• Git / GitHub – for version control, code collaboration, project management, and task tracking

• Visual Studio Code – IDE used for developing the AgriScan AI codebase and pushing code fragments to GitHub

E.Support tools

• Grammarly – for formatting documentation
• Microsoft Word – for documenting the project

• Canva – for creating a poster

9.Problem Definition

Diagnosing plant foliar diseases from leaf images is important. It requires time and resources.

In South Africa and other developing areas, there are not enough agricultural experts and plant pathologists to do this work in time.

In farming communities and smallholder farms, the problem is even more pronounced because the few available experts are overworked, leading to mistakes, delays in treating crops, and missed disease detection, which cause significant crop losses.

This problem is very critical. It must be addressed adequately because plant diseases can cause severe economic losses for farmers, reduce food security, and damage the environment through excessive pesticide use. Early detection of plant diseases and more accurate disease categorization can improve crop yields and farmers' livelihoods by supporting better agricultural decision-making. The proposed solution, AgriScan AI, uses thePlantVillage dataset and AI, ML, and DL techniques to analyze and classify plant leaf images across thirty-eight disease classes. This solution will help agricultural specialists reduce their workload, connect farmers with experts, and improve decision-making in crop disease management.

THEORETICAL ASPECTS

1. Machine Learning Approach

We have adopted a supervised machine learning approach to create AgriScan AI. By using Kerasutils function, image_dataset_from_directory, we implemented a multi class image classification task, as our model was designed to train on our labeled data inferred from their respective folders, acting as our truth labels.

```python
#Function to load the dataset
def load_datasets():
    """Load and preprocess train/val/test datasets. Returns datasets and class names."""
    base_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")
    train_dir, test_dir = os.path.join(base_dir, "train"), os.path.join(base_dir, "test")

    for d in [base_dir, train_dir, test_dir]:
        if not os.path.exists(d):
            raise FileNotFoundError(f"Directory not found: {d}")

    class_names = sorted([f for f in os.listdir(train_dir) if os.path.isdir(os.path.join(train_dir, f))])
    if len(class_names) == 0 or len(class_names) != len(set(class_names)):
        raise ValueError("Invalid or duplicate class names in training directory")

    print(f"Found {len(class_names)} disease classes")

    train_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
        shuffle=True, seed=SEED, validation_split=0.2, subset="training")

    val_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
        shuffle=False, seed=SEED, validation_split=0.2, subset="validation")

    test_ds = image_dataset_from_directory(test_dir, labels="inferred", label_mode="categorical",
        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE, shuffle=False)

    test_class_names = sorted([f for f in os.listdir(test_dir) if os.path.isdir(os.path.join(test_dir, f))])
    if set(test_class_names) != set(class_names):
        raise ValueError(f"Class mismatch between train and test directories")

    rescale = layers.Rescaling(1.0 / 255.0)
    augment = Sequential([layers.RandomFlip("horizontal"), layers.RandomZoom(0.1), layers.RandomContrast(0.1)], name="augment")

    train_ds = train_ds.map(lambda x, y: (rescale(augment(x, training=True)), y), num_parallel_calls=AUTOTUNE).cache().prefetch(AUTOTUNE)
    val_ds = val_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    test_ds = test_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
```

Figure 1: Multi-class plant disease classification task

Reference: load_datasets() function in AgriScanAI.py (lines 37-74)

The model was implemented using a Convolutional Neural Network (CNN) built with a Keras Sequential classifier and optimized with the Adam algorithm and categorical cross-entropy loss.

```python
#Function to build the CNN model
def build_model(class_names):
    """Build and compile CNN model."""
    model = Sequential(name="agriscan_cnn")
    model.add(layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)))

    filters = 32
    for _ in range(3):
        model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
        model.add(MaxPooling2D(pool_size=(2, 2)))
        filters *= 2
    model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Flatten())
    model.add(Dense(256, activation="relu"))
    model.add(Dropout(0.5))
    model.add(Dense(len(class_names), activation="softmax"))

    model.compile(optimizer=Adam(learning_rate=0.001), loss="categorical_crossentropy",
        metrics=["accuracy", metrics.Precision(name="precision"), metrics.Recall(name="recall")])
    model.summary()
    return model
```

Figure 2: The model was implemented using a Convolutional Neural Network (CNN)

Reference: build model() function in AgriScanAI.py (lines 78-99)

Our model uses the PlantVillage dataset to train via supervised learning and assess itself from the inferred truths.



Figure 3: The model uses the PlantVillage dataset

Reference: Dataset directory structure in load_datasets() function

After which it outputs for each data point a class via one of the 38 classes identified from our dataset.



```python

#Function to predict disease from image
def predict_image(image_input, model=None, class_names=None):
    """Predict disease from image (path, file object, PIL Image, or numpy array)."""
    if model is None or class_names is None:
        model, class_names = load_model()

    resolved_path = None
    img_array = None

    if isinstance(image_input, str):
        resolved_path = validate_path(image_input)
        if not resolved_path:
            raise FileNotFoundError(f"Image not found: {image_input}")
        img = tf.keras.utils.load_img(resolved_path, target_size=IMAGE_SIZE, color_mode="rgb", interpolation="bilinear")
        img_array = tf.keras.utils.img_to_array(img, dtype="float32")
    elif hasattr(image_input, "read"):
        pil_img = Image.open(image_input)
        if pil_img.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {pil_img.mode}")
        if pil_img.mode != "RGB":
            pil_img = pil_img.convert("RGB")
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, Image.Image):
        if image_input.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {image_input.mode}")
        pil_img = image_input.convert("RGB") if image_input.mode != "RGB" else image_input
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, np.ndarray):
        img_array = image_input.astype("float32")
    else:
        raise TypeError(f"Unsupported input type: {type(image_input).__name__}")

    if img_array is None or len(img_array.shape) != 3 or img_array.shape[2] not in [1, 3, 4]:
        raise ValueError(f"Invalid image array shape: {img_array.shape if img_array is not None else 'None'}")

    if img_array.shape[2] == 1:
        img_array = np.repeat(img_array, 3, axis=2)
    elif img_array.shape[2] == 4:
        img_array = img_array[:, :, :3]

    if img_array.max() > 1.0:
        img_array = img_array / 255.0

    if img_array.min() < 0.0 or img_array.max() > 1.0:
        raise ValueError(f"Invalid pixel range after normalization: [{img_array.min():.4f}, {img_array.max():.4f}]")

    if img_array.shape[:2] != IMAGE_SIZE:
        pil_img = Image.fromarray((img_array * 255).astype(np.uint8))
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32) / 255.0

    img_array = np.expand_dims(img_array, axis=0)
    probs = model.predict(img_array, verbose=0)[0]

    if not np.all(np.isfinite(probs)) or len(probs) != len(class_names):
        raise ValueError("Invalid model predictions")

    top_idx = int(np.argmax(probs))
    top_class = class_names[top_idx]
    top_conf = float(probs[top_idx])

    sorted_idxs = np.argsort(probs)[::-1]
    top_three = [{"RANK": i + 1, "CLASS_NAME": class_names[int(sorted_idxs[i])],
                  "CONFIDENCE_SCORE": float(probs[int(sorted_idxs[i])]),
                  "CONFIDENCE_PERCENTAGE": f"{probs[int(sorted_idxs[i])]:.2%}"}
                 for i in range(min(3, len(sorted_idxs)))]

    disease_info = get_disease_information(top_class)

    if top_conf >= 0.85:
        reliability = "High confidence (probability estimate). Field verification still recommended."
    elif top_conf >= 0.50:
        reliability = "Moderate confidence. Inspect symptoms carefully and verify with secondary predictions."
    else:
        reliability = "Low confidence. Symptoms may be ambiguous or at early stage. Field confirmation strongly recommended."

    return {
        "SUCCESS": True,
        "IMAGE_PATH": resolved_path or "In-Memory Image",
        "PREDICTED_CLASS": top_class,
        "CONFIDENCE_SCORE": top_conf,
        "CONFIDENCE_PERCENTAGE": f"{top_conf:.2%}",
        "TOP_THREE_PREDICTIONS": top_three,
        "DISEASE_INFORMATION": disease_info,
        "RELIABILITY_NOTE": reliability
    }
```

```python
#Function to calculate class weights
def calculate_class_weights(train_ds, class_names):
    """Calculate balanced class weights from training dataset labels."""
    print("Calculating class weights from training data...")
    y_train = np.concatenate([np.argmax(labels.numpy(), axis=1) for _, labels in train_ds])

    class_weights = compute_class_weight(class_weight='balanced', classes=np.arange(len(class_names)), y=y_train)
    class_weight_dict = dict(enumerate(class_weights))

    print(f"Class Weight Statistics: Min: {min(class_weights):.4f}, Max: {max(class_weights):.4f}, Mean: {np.mean(class_weights):.4f}")

    for class_name in ['Apple___Apple_scab', 'Apple___Cedar_apple_rust', 'Potato___healthy', 'Raspberry___healthy', 'Orange___Haunglongbing_(Citrus_greening)']:
        if class_name in class_names:
            print(f"  {class_name}: {class_weights[class_names.index(class_name)]:.4f}")

    return class_weight_dict

```

Figure 4: Outputs for each data class via one of the 38 classes

Reference: predict_image() function output structure (lines 277-360)

This approach was selected for its proven efficiency in learning spatial features from image data, making it a strong choice for our plant disease classification task. To handle dataset imbalance, we implemented class weighting to ensure minority classes receive appropriate attention during training.

2. Data

AgriScan AI employed the PlantVillage dataset sourced from the Kaggle dataset library and focused on the plant foliar disease classification task. The folder structure includes train and test folders, each with 38 class folders representing different plant diseases across multiple crop types. In total, there are approximately 54,000+ images, resized to 128 X 128 pixels for model training.

```python
#Function to load the dataset
def load_datasets():
    """Load and preprocess train/val/test datasets. Returns datasets and class names."""
    base_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")
    train_dir, test_dir = os.path.join(base_dir, "train"), os.path.join(base_dir, "test")

    for d in [base_dir, train_dir, test_dir]:
        if not os.path.exists(d):
            raise FileNotFoundError(f"Directory not found: {d}")

```

Figure 5: The PlantVillage dataset was sourced from the Kaggle dataset library and focused on the plant foliar disease classification task

Reference: Dataset structure in load_datasets() function (lines 39-46)

For training purposes, 20% of the training data was set aside for validation to track metrics per epoch.

```
52    train_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
53        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
54        shuffle=True, seed=SEED, validation_split=0.2, subset="training")
55
56    val_ds = image_dataset_from_directory(train_dir, labels="inferred", label_mode="categorical",
57        class_names=class_names, color_mode="rgb", image_size=IMAGE_SIZE, batch_size=BATCH_SIZE,
58        shuffle=False, seed=SEED, validation_split=0.2, subset="validation")
```

Figure 6: 20% of the training data was set aside for validation

Reference: Validation split in image_dataset_from_directory (lines 52-58)

In the codebase, all images were resized to 128 X 128 pixels to improve efficiency and normalized to [0, 1]. To improve the robustness of training, we applied data augmentation on the training set only, such as horizontal flips, zooms, and contrast adjustments. We only normalized the validation and test sets.

```
165  #Function to calculate class weights
166  def calculate_class_weights(train_ds, class_names):
167      """Calculate balanced class weights from training dataset labels."""
168      print("Calculating class weights from training data...")
169      y_train = np.concatenate([np.argmax(labels.numpy(), axis=1) for _, labels in train_ds])
170
171      class_weights = compute_class_weight(class_weight='balanced', classes=np.arange(len(class_names)), y=y_train)
172      class_weight_dict = dict(enumerate(class_weights))
173
174      print(f"Class Weight Statistics: Min: {min(class_weights):.4f}, Max: {max(class_weights):.4f}, Mean: {np.mean(class_weights):.4f}")
175
176      for class_name in ['Apple___Apple_scab', 'Apple___Cedar_apple_rust', 'Potato___healthy', 'Raspberry___healthy', 'Orange___Haunglongbing_(Citrus_greening)']:
177          if class_name in class_names:
178              print(f"  {class_name}: {class_weights[class_names.index(class_name)]:.4f}")
179
180      return class_weight_dict
```

```
67   rescale = layers.Rescaling(1.0 / 255.0)
68   augment = Sequential([layers.RandomFlip("horizontal"), layers.RandomZoom(0.1), layers.RandomContrast(0.1)], name="augment")
69
70   train_ds = train_ds.map(lambda x, y: (rescale(augment(x, training=True)), y), num_parallel_calls=AUTOTUNE).cache().prefetch(AUTOTUNE)
71   val_ds = val_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
72   test_ds = test_ds.map(lambda x, y: (rescale(x), y), num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
73
```

Figure 7: In the codebase, all images were resized to 128 X 128 pixels to improve efficiency and normalized to [0, 1]

Reference: Preprocessing in load_datasets() function (lines 67-72)

The dataset exhibits significant class imbalance, which we addressed by computing balanced class weights using scikit-learn's compute_class_weight function.

Reference: calculate_class_weights() function (lines 166-180)

3. Model

AgriScan AI uses a Convolutional Neural Network with a Keras Sequential classifier. We implemented a 4-block system of Convolutional and Pooling layers using Keras. using Conv2D and MaxPooling2D classifiers. By reducing the spatial data by half and increasing feature detectors by 2, we ensured that each block learns from the preceding block.

```
77  #Function to build the CNN model
78  def build_model(class_names):
79      """Build and compile CNN model."""
80      model = Sequential(name="agriscan_cnn")
81      model.add(layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)))
82
83      filters = 32
84      for _ in range(3):
85          model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
86          model.add(MaxPooling2D(pool_size=(2, 2)))
87          filters *= 2
88      model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
89      model.add(MaxPooling2D(pool_size=(2, 2)))
90
91      model.add(Flatten())
92      model.add(Dense(256, activation="relu"))
93      model.add(Dropout(0.5))
94      model.add(Dense(len(class_names), activation="softmax"))
95
96      model.compile(optimizer=Adam(learning_rate=0.001), loss="categorical_crossentropy",
97          metrics=["accuracy", metrics.Precision(name="precision"), metrics.Recall(name="recall")])
98      model.summary()
99      return model
```

Figure 8: Implementation of a 4-block system of Convolutional and Pooling layers using Keras.layers' Conv2D and MaxPooling2D classifiers Reference: build_model() function (lines 78-99)

A Flatten layer was used to condense the outputs from the blocks into 1D feature vectors for our fully connected dense layer. The dense layer adopts the "ReLU" activation function to evaluate our model's decision. A dropout layer of 50% was employed to promote regularization and avoid overfitting before the final dense layer for outputting results for each image. The model was then compiled using the Adam optimizer and metrics to be tracked and assigned within.

```
91      model.add(Flatten())
92      model.add(Dense(256, activation="relu"))
93      model.add(Dropout(0.5))
94      model.add(Dense(len(class_names), activation="softmax"))
```

Figure 9: *A Flatten layer was used to condense the outputs*

Reference: build_model() function architecture (lines 91-94)

The model was then set to train for 6 epochs with early stopping using the training dataset and validation dataset to track progress per epoch.



```python
#Function to train model with class weights
def train_model_with_class_weights(epochs=6):
    """Train model with class weights to handle imbalance."""
    if not isinstance(epochs, int) or epochs <= 0:
        raise ValueError(f"Epochs must be a positive integer, got: {epochs}")

    os.makedirs("models", exist_ok=True)
    os.makedirs("results", exist_ok=True)

    train_ds, val_ds, test_ds, class_names = load_datasets()
    model = build_model(class_names)
    class_weight_dict = calculate_class_weights(train_ds, class_names)

    with open("models/class_names_class_weighted.json", "w", encoding="utf-8") as f:
        json.dump(class_names, f, indent=4)

    with open("results/class_weights.json", "w", encoding="utf-8") as f:
        json.dump({str(k): float(v) for k, v in class_weight_dict.items()}, f, indent=4)

    callbacks = [
        EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True, verbose=1),
        ModelCheckpoint(filepath="models/class_weighted_model.keras", monitor="val_accuracy", save_best_only=True, verbose=1),
        ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-5, verbose=1)
    ]

    print(f"\nTraining with class weights for {epochs} epochs...")
    history = model.fit(train_ds, validation_data=val_ds, epochs=epochs, class_weight=class_weight_dict, callbacks=callbacks, verbose=1)

    hist_dict = {k: [float(v) for v in vals] for k, vals in history.history.items()}
    with open("results/training_history_class_weighted.json", "w", encoding="utf-8") as f:
        json.dump(hist_dict, f, indent=4)

    plot_training_history(hist_dict, "(Class Weighted)", "results/accuracy_loss_curves_class_weighted.png")

    checkpoint_path = "models/class_weighted_model.keras"
    if not os.path.exists(checkpoint_path):
        raise FileNotFoundError(f"Training completed but checkpoint not found at {checkpoint_path}")

    model = tf.keras.models.load_model(checkpoint_path)
    print("Loaded best class-weighted model checkpoint for evaluation")

    if model.output_shape[-1] != len(class_names):
        raise ValueError(f"Model output dimension ({model.output_shape[-1]}) does not match number of classes ({len(class_names)})")

    evaluate_model(model, test_ds, class_names, "(Class Weighted)", "_class_weighted")
    return True
```

Figure 10: The model was then set to train for 6 epochs with early stopping

Reference: train_model_with_class_weights() function (lines 184-228)

After which, the model was then used to evaluate and predict on the unseen test data.



```python
#Function to evaluate the model
def evaluate_model(model, test_ds, class_names, title_suffix, output_suffix):
    """Evaluate model, generate confusion matrix and classification report."""
    loss, acc, prec, rec = model.evaluate(test_ds, verbose=1)[:4]
    print(f"Test - Loss: {loss:.4f}, Acc: {acc:.4f}, Prec: {prec:.4f}, Rec: {rec:.4f}")

    predictions = model.predict(test_ds, verbose=1)
    if not np.all(np.isfinite(predictions)):
        raise ValueError("Model predictions contain non-finite values (NaN or Inf)")

    y_pred = np.argmax(predictions, axis=1)
    y_true = np.concatenate([np.argmax(labels.numpy(), axis=1) for _, labels in test_ds])

    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=class_names)
    fig, ax = plt.subplots(figsize=(16, 14))
    disp.plot(ax=ax, cmap="Blues", values_format="d", xticks_rotation=90)
    plt.title(f"AgriScan AI - 38 Class Plant Disease Confusion Matrix {title_suffix}")
    plt.tight_layout()
    plt.savefig(f"results/confusion_matrix{output_suffix}.png", dpi=300)
    plt.close(fig)
    print("Confusion matrix saved")

    report = classification_report(y_true, y_pred, target_names=class_names, digits=4, zero_division=0)
    print(f"\nClassification Report {title_suffix}:")
    print(report)

    report_dict = classification_report(y_true, y_pred, target_names=class_names, digits=4, output_dict=True, zero_division=0)
    with open(f"results/classification_report{output_suffix}.json", "w", encoding="utf-8") as f:
        json.dump(report_dict, f, indent=4)
    print("Classification report saved")
```

Figure 11: The model was then used to evaluate and predict on the unseen test data

Reference: evaluate_model() function (lines 133-162)

4. Solution Techniques

- The team used a weighted-scale decision method to evaluate several projects ideas based on feasibility and social impact. The highest-scoring idea, AgriScan AI, was selected for implementation.

- Each team member contributed modular Python functions based on their assigned task. Various code versions were evaluated and merged via GitHub branches.

- All images from the PlantVillage dataset were resized to 128 x 128 pixels to reduce computational cost and training time per epoch without sacrificing key spatial features.
- Every image was normalized from an RGB range of [0-255] to [0-1] using Keras'layers.Rescaling function to prevent model bias towards large pixel values.

- We applied a fixed global seed on all random generators used (Python, Numpy, and TensorFlow) to ensure reproducible results per run and allow for easier evaluation of the model per run.

- The training dataset was further augmented using the Sequential model, with random horizontal flips, zooms, and contrast adjustments applied to random images, to improve the robustness of the training process of our CNN model and enhance generalization.

- Datasets were cached in RAM and prefetched using tensorflow.data.AUTOTUNE efficiency algorithm to enable parallel data loading and maximize CPU
utilization, thus improving training efficiency.

- We used TensorFlow data AUTOTUNE to evaluate local machine CPU features and load in parallel to maximize compute power efficiency.

- The convolution layer used the "same" padding to include edge pixels during feature extraction and ensure a uniform spatial coverage.

- The number of filters was doubled after each Convolution-Pooling block to compensate for spatial reduction caused by the Pooling layer, and allow for learning of complex abstractions.
- Our model adopted the ReLU activation to introduce a non-linear learning pattern and improve the efficiency of the learning process.

- A dropout of 50% was applied to randomly deactivate half of the neurons during training to prevent feature memorization, causing overfitting and thus promoting generalization.

- The model was compiled using the Adam optimizer algorithm to ensure our model gains adaptive learning capabilities.

- The training data was split, and 20% set aside for validation to be used for tracking the learning process per epoch cycle and help detect underfitting or overfitting in training.

- We implemented class weighting to handle dataset imbalance, ensuring minority classes receive appropriate attention during training.

- We ensured to implement a confidence score when predicting every single image to help users measure how certain the model is about its diagnosis – essential for real-world application of AgriScan AI.

- We implemented reliability indicators categorizing predictions as high confidence (≥85%), moderate confidence (50-85%), or low confidence (<50%) with appropriate recommendations.

- Python's built-in error handler was integrated into all functions to identify, isolate, and handle runtime errors efficiently, improving debugging and maintainability.

- We implemented three training callbacks: EarlyStopping to prevent overfitting, ModelCheckpoint to save the best model, and ReduceLROnPlateau to adjust learning rate during training.

5. Deep Learning

AgriScan AI applies Deep Learning techniques through a Convolutional Neural Network using the Keras Sequential classifier. CNNs are well-suited for image-based tasks because they can process two-dimensional spatial data and automatically extract meaningful visual features, making them an ideal choice for agricultural image classification tasks.

The CNN architecture consists of 4 Convolution-Pooling blocks, allowing the model to progressively learn and represent more complex spatial features at each block level.

```
83      filters = 32
84      for _ in range(3):
85          model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
86          model.add(MaxPooling2D(pool_size=(2, 2)))
87          filters *= 2
88      model.add(Conv2D(filters, (3, 3), padding="same", activation="relu"))
89      model.add(MaxPooling2D(pool_size=(2, 2)))
90
91      model.add(Flatten())
92      model.add(Dense(256, activation="relu"))
93      model.add(Dropout(0.5))
```

Reference: Architecture in build_model() function (lines 84-89)

ReLU activation functions were applied to introduce non-linear learning behaviour, enabling the network to capture irregular disease patterns.

A Flatten layer was used to summarize 2D spatial features to 1D feature vectors, which were then passed into a Dense layer to perform high-level reasoning and classification.

Finally, a Dropout layer was integrated to promote regularization, reduce overfitting, and enhance the model's ability to generalize to unseen plant leaf images.

The model employs hierarchical feature learning, where lower layers learn simple features like edges and colors, middle layers learn complex textures and patterns, and deeper layers learn disease-specific characteristics. This automatic feature extraction eliminates the need for manual feature engineering.

Reference: Convolutional blocks in build_model() function (lines 84-89)

6. Other Features

AgriScan AI integrated several features into its solution - the ability to predict on a single plant leaf image through multiple input types (file paths, file objects, PIL Images, or NumPy arrays). This feature is particularly useful for real-world applications and promotes ease of use for both technical and non- technical users.

The system provides a command-line interface for easy use and a programmatic API for integration with other systems.

For each prediction, the system presents the model's predicted disease class, the confidence score, the top 3 predictions with probabilities, and comprehensive disease information including symptoms, prevention methods, and recommended actions.

The system also generates comprehensive evaluation visualizations including training history plots (loss and accuracy curves), confusion matrix heatmaps, and detailed classification reports.

We integrated a disease knowledge base that provides expert-curated information for each of the 38 disease classes, making the system more than just a classifier - it becomes an educational and decision-support tool.

The system categorizes predictions based on confidence scores and provides appropriate reliability indicators. For low-confidence predictions, special warnings are displayed advising users not to rely solely on the prediction and to seek field verification.

Reference: Reliability assessment in predict_image() function (lines 344-349)



```
#Function to predict disease from image
def predict_image(image_input, model=None, class_names=None):
    """Predict disease from image (path, file object, PIL Image, or numpy array)."""
    if model is None or class_names is None:
        model, class_names = load_model()

    resolved_path = None
    img_array = None

    if isinstance(image_input, str):
        resolved_path = validate_path(image_input)
        if not resolved_path:
            raise FileNotFoundError(f"Image not found: {image_input}")
        img = tf.keras.utils.load_img(resolved_path, target_size=IMAGE_SIZE, color_mode="rgb", interpolation="bilinear")
        img_array = tf.keras.utils.img_to_array(img, dtype="float32")
    elif hasattr(image_input, "read"):
        pil_img = Image.open(image_input)
        if pil_img.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {pil_img.mode}")
        if pil_img.mode != "RGB":
            pil_img = pil_img.convert("RGB")
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, Image.Image):
        if image_input.mode not in ["RGB", "RGBA", "L", "P"]:
            raise ValueError(f"Unsupported image mode: {image_input.mode}")
        pil_img = image_input.convert("RGB") if image_input.mode != "RGB" else image_input
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32)
    elif isinstance(image_input, np.ndarray):
        img_array = image_input.astype("float32")
    else:
        raise TypeError(f"Unsupported input type: {type(image_input).__name__}")

    if img_array is None or len(img_array.shape) != 3 or img_array.shape[2] not in [1, 3, 4]:
        raise ValueError(f"Invalid image array shape: {img_array.shape if img_array is not None else 'None'}")

    if img_array.shape[2] == 1:
        img_array = np.repeat(img_array, 3, axis=2)
    elif img_array.shape[2] == 4:
        img_array = img_array[:, :, :3]

    if img_array.max() > 1.0:
        img_array = img_array / 255.0

    if img_array.min() < 0.0 or img_array.max() > 1.0:
        raise ValueError(f"Invalid pixel range after normalization: [{img_array.min():.4f}, {img_array.max():.4f}]")

    if img_array.shape[:2] != IMAGE_SIZE:
        pil_img = Image.fromarray((img_array * 255).astype(np.uint8))
        img_array = np.array(pil_img.resize(IMAGE_SIZE, Image.Resampling.BILINEAR), dtype=np.float32) / 255.0

    img_array = np.expand_dims(img_array, axis=0)
    probs = model.predict(img_array, verbose=0)[0]

    if not np.all(np.isfinite(probs)) or len(probs) != len(class_names):
        raise ValueError("Invalid model predictions")

    top_idx = int(np.argmax(probs))
    top_class = class_names[top_idx]
    top_conf = float(probs[top_idx])

    sorted_idxs = np.argsort(probs)[::-1]
    top_three = [{"RANK": i + 1, "CLASS_NAME": class_names[int(sorted_idxs[i])],
                  "CONFIDENCE_SCORE": float(probs[int(sorted_idxs[i])]),
                  "CONFIDENCE_PERCENTAGE": f"{probs[int(sorted_idxs[i])]:.2%}"}
                 for i in range(min(3, len(sorted_idxs)))]

    disease_info = get_disease_information(top_class)

    if top_conf >= 0.85:
        reliability = "High confidence (probability estimate). Field verification still recommended."
    elif top_conf >= 0.50:
        reliability = "Moderate confidence. Inspect symptoms carefully and verify with secondary predictions."
    else:
        reliability = "Low confidence. Symptoms may be ambiguous or at early stage. Field confirmation strongly recommended."

    return {
        "SUCCESS": True,
        "IMAGE_PATH": resolved_path or "In-Memory Image",
        "PREDICTED_CLASS": top_class,
        "CONFIDENCE_SCORE": top_conf,
        "CONFIDENCE_PERCENTAGE": f"{top_conf:.2%}",
        "TOP_THREE_PREDICTIONS": top_three,
        "DISEASE_INFORMATION": disease_info,
        "RELIABILITY_NOTE": reliability
    }
```

Figure 12: The predict_image function accepts multiple input types and displays comprehensive results including disease information, confidence scores, and reliability indicators

Reference: predict_image() function (lines 277-360)

References

1. Chollet, F., 2021. *Deep Learning with Python*. 2nd edn. New York: Manning Publications.

2. Food and Agriculture Organization (FAO), n.d. *About FAO's work on plant production and protection*. [online] Available at: <fao.org> [Accessed 2 October 2026].

3. Food and Agriculture Organization (FAO), n.d. *Agricultural Statistics*. [online] Available at: <fao.org> [Accessed 2 October 2026].

4. Géron, A., 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 3rd ed. Sebastopol: O'Reilly Media.

5. Goodfellow, I., Bengio, Y. and Courville, A., 2016. *Deep Learning*. Cambridge, MA: MIT Press.
6. Howard, J. and Gugger, S., 2020. *Deep Learning for Coders with fastai and PyTorch: AI Applications Without a PhD*. Sebastopol: O'Reilly Media.

7. Hughes, D.P. and Salathé, M., 2015. *An open access repository of images on plant health*. arXiv preprint arXiv:1511.08060. Available at: <arxiv.org>

8. International Plant Protection Convention (IPPC) Secretariat, 2026. *International Day of Plant Health 2026 spotlights plant biosecurity as a foundation for global food security*. Rome: FAO. Available at: <ippc.int> [Accessed 20 September 2026].

9. Keras, n.d. *Keras Documentation*. [online] Available at: <keras.io> [Accessed 2 November 2026].

10. Mohanty, S.P., Hughes, D.P. and Salathé, M., 2016. Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, p.1419. Available at: <doi.org>.

11. PlantVillage, n.d. *PlantVillage Dataset*. Hugging Face. Available at: <huggingface.co> [Accessed 15 August 2026].

12. Scikit-learn, n.d. *Scikit-learn Documentation*. [online] Available at: <scikit-learn.org> [Accessed 2 August 2026]

13. TensorFlow, n.d. *TensorFlow Documentation*. [online] Available at: <tensorflow.org> [Accessed July 2026].

GRAMMARLY REPORT

AgriScan AI Plant Disease Diagnostics

by MATHEBULA NHLAYISEKO

General metrics

25,993	3,559	378	14 min 14 sec	27 min 22 sec
characters	words	sentences	reading time	speaking time

Score



This text scores better than 87% of all texts checked by Grammarly

Writing Issues

88	51	37
Issues left	Critical	Advanced

Plagiarism



0
sources

1% of your text matches 0 sources on the web or